

GeGraphs v0.2.9 Tutorial

Gene-centric visualization and structural variant exploration

D. Llugiqi

University of Geneva (UNIGE)

Master in Bioinformatics and Data Analysis in Biology (BIADB)

2026-08-24

Contents

1	Introduction	2
2	Installation	3
2.1	Install the package	3
2.2	Main dependencies	3
3	Input data	3
4	Core visualization functions	4
4.1	plot_gene()	4
4.1.1	Purpose	4
4.1.2	Minimal example	4
4.1.3	Adding controls and SV calls	4
4.1.4	Main arguments	5
4.1.5	Example output	6
4.2	plot_region()	6
4.2.1	Purpose	6
4.2.2	Minimal example	6
4.2.3	With controls and SV tracks	7
4.2.4	Example output	7
5	Reading the output	8
5.1	Gene annotation	8
5.2	Raw coverage	8
5.3	Normalized coverage	8
5.4	Structural variant tracks	8
6	Choosing the coverage bin size	9
7	Exploring genes affected by structural variants	9
7.1	get_genes_under_sv()	9
7.2	Gene context	9

8	Panel-based SV analysis	10
8.1	<code>panels_overlap_df()</code>	10
8.1.1	Basic use	10
8.1.2	Generate plots automatically	11
9	Complete example workflow	11
10	Good practices and troubleshooting	12
10.1	BAM index not found	12
10.2	Gene not found	12
10.3	MANE / GENCODE messages	12
10.4	Non-standard chromosomes	12
10.5	Plot contains too many coverage points	13
10.6	Large BED or VCF files	13
11	Summary	13
12	Citation	13
13	Session Info	13

1 Introduction

GeGraphs is an R package developed to facilitate the visual inspection and exploration of structural variants (SVs) in genomic data.

The package brings together three types of information that are usually inspected separately:

- gene annotations from GENCODE,
- sequencing coverage from BAM files,
- structural variant calls from BED or VCF files.

The main objective is to provide a simple gene-centred view that can be used to inspect whether an SV call is consistent with the local genomic context and coverage pattern.

Version 0.2.9 includes the core visualization functions from previous versions and adds utilities for exploring genes affected by SVs, examining neighbouring genes, and intersecting SV calls with gene panels.

The main functions presented in this tutorial are:

- `plot_gene()` — visualize a gene together with coverage and SV calls,
- `plot_region()` — visualize an arbitrary genomic interval,
- `get_genes_under_sv()` — identify genes overlapped by structural variants,
- `get_gene_context()` and `get_gene_context_for_gene()` — inspect neighbouring genes,
- `panels_overlap_df()` — identify panel genes overlapped by SVs and optionally generate plots.

2 Installation

2.1 Install the package

If the package source is available locally:

```
devtools::install("/path/to/GeGraphs")
```

Then load the package:

```
library(GeGraphs)
packageVersion("GeGraphs")
```

```
## [1] '0.2.9'
```

A successful installation should return version 0.2.9.

2.2 Main dependencies

GeGraphs relies on several Bioconductor packages for genomic ranges, BAM processing and genomic visualization.

If required, these packages can be loaded explicitly:

```
library(GenomicRanges)
library(IRanges)
library(Rsamtools)
library(rtracklayer)
library(Gviz)
library(S4Vectors)
```

3 Input data

The visualization functions require standard genomic file formats.

For a typical analysis, the user provides:

- one BAM file for the sample of interest,
- the corresponding BAM index (.bam.bai),
- optional BAM files for controls or family members,
- optional BED or VCF files containing structural variant calls.

Example file paths:

```
sample_bam <- "path/to/sample.bam"

control_bams <- c(
  Control_1 = "path/to/control_1.bam",
  Control_2 = "path/to/control_2.bam"
)

sv_files <- list(
  Delly = "path/to/sample.delly.vcf",
```

```
Dysgu = "path/to/sample.dysgu.vcf"  
)
```

BAM files must be indexed before they are used.

```
samtools index sample.bam
```

4 Core visualization functions

4.1 plot_gene()

4.1.1 Purpose

`plot_gene()` creates a multi-track figure centred on a selected gene.

Depending on the arguments provided, the plot can contain:

1. a genomic coordinate axis,
2. the gene structure,
3. raw sequencing coverage,
4. normalized coverage for the sample of interest and controls,
5. one or more structural variant tracks.

Gene coordinates are obtained from the annotation supplied with the package. The reference genome is selected with the `genome` argument.

4.1.2 Minimal example

```
plot_gene(  
  gene = "ERGIC1",  
  bam_file = sample_bam,  
  genome = "hg19",  
  out = "ERGIC1_plot.pdf"  
)
```

This is the simplest use of `plot_gene()`: a gene and one BAM file.

4.1.3 Adding controls and SV calls

```
plot_gene(  
  gene = "ERGIC1",  
  bam_file = sample_bam,  
  control_bams = control_bams,  
  vcf_list = sv_files,  
  genome = "hg19",  
  flank = 5000,  
  show_gene = TRUE,  
  show_cov = TRUE,  
  show_sv = TRUE,  
  bin_size = 2000,
```

```

    title = "ERGIC1 - coverage and structural variants",
    out = "ERGIC1_full.pdf"
)

```

The names used in `control_bams` and `vcf_list` are used as labels in the plot.

4.1.4 Main arguments

Argument	Description
<code>gene</code>	Gene symbol to visualize
<code>bam_file</code>	BAM file for the sample of interest
<code>control_bams</code>	Optional named vector or list of control BAM files
<code>vcf_list</code>	Optional named list of VCF files
<code>bed_list</code>	Optional named list of BED files
<code>genome</code>	Reference genome, "hg19" or "hg38"
<code>flank</code>	Number of base pairs displayed upstream and downstream of the gene
<code>show_gene</code>	Display the gene annotation track
<code>show_cov</code>	Display coverage tracks
<code>show_sv</code>	Display structural variant tracks
<code>bin_size</code>	Bin size used for normalized coverage
<code>title</code>	Optional custom title
<code>out</code>	Output file name

4.1.5 Example output

The following figure illustrates the type of output produced by `plot_gene()`. It combines the gene annotation, raw read coverage, normalized coverage and an SV call in a single gene-centred view.

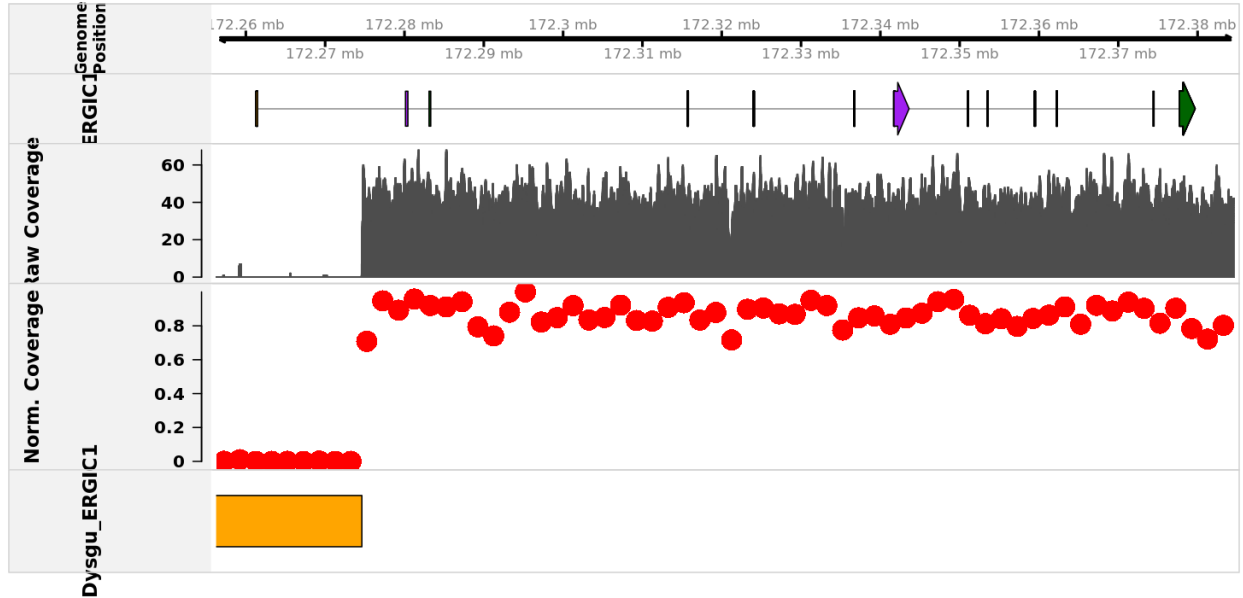


Figure 1 – Example of a gene-centred visualization with `plot_gene()`

The gene annotation is shown together with raw and normalized sequencing coverage and the structural variant track. This view allows the different sources of information to be inspected together when reviewing a candidate SV.

4.2 `plot_region()`

4.2.1 Purpose

`plot_region()` uses the same visualization framework as `plot_gene()`, but the region is defined directly by chromosome coordinates.

This is useful when:

- the SV extends beyond a single gene,
- several neighbouring genes need to be inspected together,
- the region of interest is intergenic,
- a broader copy-number pattern needs to be visualized.

4.2.2 Minimal example

```
plot_region(  
  chrom = "5",  
  start = 172260000,  
  end = 172360000,  
  bam_file = sample_bam,  
  genome = "hg19",
```

```

    out = "chr5_region.pdf"
)

```

4.2.3 With controls and SV tracks

```

plot_region(
  chrom = "5",
  start = 172260000,
  end = 172360000,
  bam_file = sample_bam,
  control_bams = control_bams,
  vcf_list = sv_files,
  genome = "hg19",
  show_cov = TRUE,
  show_sv = TRUE,
  bin_size = 5000,
  title = "Chromosome 5 region",
  out = "chr5_region_full.pdf"
)

```

4.2.4 Example output

For larger intervals, `plot_region()` provides a broader view of the coverage pattern. In trio analyses, the proband, mother and father are shown in different colours, making it easier to compare their normalized coverage and visually assess the segregation of a candidate event.

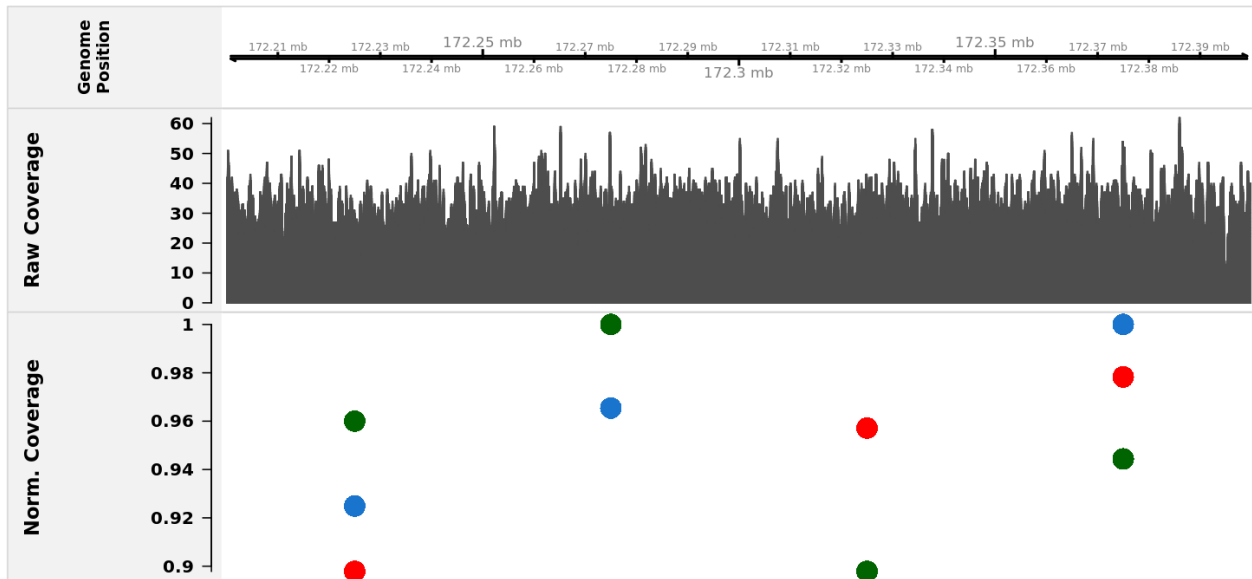


Figure 2 – Example of a region-based visualization with `plot_region()`

Raw coverage is displayed across a broad genomic interval, while normalized coverage summarizes the sample of interest and controls using larger bins. This view is useful for examining coverage patterns beyond the limits of a single gene.

5 Reading the output

A GeGraphs figure is designed to combine complementary information in a single view.

5.1 Gene annotation

The gene track represents the annotated structure of the selected gene. Depending on the available annotation, coding regions and untranslated regions are displayed separately.

When MANE transcript information is available and compatible with the GENCODE annotation, GeGraphs prioritizes MANE transcripts. If no matching MANE transcript is found, the package falls back to the available GENCODE annotation for the gene.

5.2 Raw coverage

The raw coverage track represents read depth across the selected locus.

A local decrease in coverage may support a deletion, while an increase may support a duplication. Raw coverage should nevertheless be interpreted carefully because sequencing depth can vary across genomic regions.

5.3 Normalized coverage

Normalized coverage is shown in bins and facilitates comparison between the sample of interest and controls.

The `bin_size` parameter determines the resolution of this track:

- smaller bins provide finer resolution,
- larger bins smooth local variation and are useful for broader regions.

5.4 Structural variant tracks

BED and VCF files can be added as separate tracks.

This makes it possible to compare calls from different algorithms directly with the coverage signal and gene annotation.

The presence of a call in the plot should not be considered sufficient evidence on its own. The purpose of the visualization is to support manual review by showing several sources of evidence together.

6 Choosing the coverage bin size

There is no single optimal `bin_size` for every locus.

A small region generally benefits from smaller bins, while larger genomic windows are easier to interpret with larger bins.

Region size	Suggested starting <code>bin_size</code>
< 50 kb	1,500–3,000 bp
50–200 kb	3,000–5,000 bp
200–500 kb	5,000–10,000 bp
> 500 kb	10,000–50,000 bp

These values are starting points rather than fixed rules. The most appropriate bin size depends on the size of the region, sequencing depth and the type of event being inspected.

7 Exploring genes affected by structural variants

7.1 `get_genes_under_sv()`

`get_genes_under_sv()` identifies genes whose genomic coordinates overlap SVs contained in a BED or VCF file.

```
genes_under <- get_genes_under_sv(  
  bed_file = "path/to/sample.filtered_DEL.bed",  
  genome = "hg19"  
)  
  
head(genes_under)
```

The returned table contains the overlapping genes and their genomic coordinates.

The result can be exported for downstream analysis:

```
write.table(  
  genes_under,  
  file = "genes_under_sv.tsv",  
  sep = "\t",  
  row.names = FALSE,  
  quote = FALSE  
)
```

7.2 Gene context

`get_gene_context()` and `get_gene_context_for_gene()` can be used to inspect genes located around a region of interest.

For example:

```
context <- get_gene_context_for_gene(
  gene = "ERGIC1",
  genome = "hg19",
  n_upstream = 2,
  n_downstream = 2
)

context
```

This is useful when an SV extends beyond one gene or when nearby genes may also be relevant.

8 Panel-based SV analysis

8.1 panels_overlap_df()

`panels_overlap_df()` intersects a gene panel with structural variant calls.

The function can be used in two ways:

- return the panel genes overlapped by SVs,
- return the overlaps and automatically generate one `plot_gene()` figure per overlapping gene.

8.1.1 Basic use

```
panel_df <- read.delim(
  "path/to/gene_panel.tsv",
  stringsAsFactors = FALSE
)

colnames(panel_df)

panel_hits <- panels_overlap_df(
  paneldf = panel_df,
  genecol = "GeneSymbol",
  bedfile = "path/to/sample.filtered_DEL.bed",
  vcffile = NULL,
  svtype = "DEL",
  genome = "hg19",
  makeplots = FALSE
)

head(panel_hits)
```

The value supplied to `genecol` must correspond to the column containing gene symbols in the panel file.

8.1.2 Generate plots automatically

```
panel_hits <- panels_overlap_df(  
  panelfdf = panel_df,  
  genecol = "GeneSymbol",  
  bedfile = "path/to/sample.filtered_DEL.bed",  
  vcffile = NULL,  
  svtype = "DEL",  
  genome = "hg19",  
  makeplots = TRUE,  
  bamfile = sample_bam,  
  controlbams = control_bams,  
  plotdir = "panel_sv_plots",  
  flank = 5000,  
  binsize = 2000  
)
```

When `makeplots = TRUE`, GeGraphs generates a gene-centred plot for each panel gene overlapping an SV.

9 Complete example workflow

A typical analysis can be organized in four steps.

```
library(GeGraphs)  
  
# 1. Define files  
sample_bam <- "path/to/sample.bam"  
  
controls <- c(  
  Control_1 = "path/to/control_1.bam",  
  Control_2 = "path/to/control_2.bam"  
)  
  
sv_files <- list(  
  Delly = "path/to/sample.delly.vcf",  
  Dysgu = "path/to/sample.dysgu.vcf"  
)  
  
# 2. Identify genes affected by SVs  
genes_under <- get_genes_under_sv(  
  vcf_file = sv_files$Dysgu,  
  genome = "hg19"  
)  
  
# 3. Inspect a selected gene  
plot_gene(  
  gene = "ERGIC1",
```

```

bam_file = sample_bam,
control_bams = controls,
vcf_list = sv_files,
genome = "hg19",
flank = 5000,
bin_size = 2000,
out = "ERGIC1_review.pdf"
)

# 4. Inspect a broader region if needed
plot_region(
    chrom = "5",
    start = 172260000,
    end = 172360000,
    bam_file = sample_bam,
    control_bams = controls,
    vcf_list = sv_files,
    genome = "hg19",
    bin_size = 10000,
    out = "chr5_review.pdf"
)

```

This workflow moves from an SV file to the genes affected by the calls and then to targeted visualization of selected candidates.

10 Good practices and troubleshooting

10.1 BAM index not found

Each BAM file must have a corresponding index.

```
samtools index sample.bam
```

10.2 Gene not found

Check that the gene symbol is correct and that the selected reference genome matches the annotation and input files.

10.3 MANE / GENCODE messages

Messages indicating that MANE transcripts were found but did not match transcript IDs in the GENCODE file are informational.

In this situation, GeGraphs uses the available GENCODE annotation instead of stopping the analysis.

10.4 Non-standard chromosomes

GeGraphs removes non-standard contigs such as random or unplaced scaffolds when preparing genomic ranges. The main analyses therefore focus on standard chromosomes.

10.5 Plot contains too many coverage points

Increase `bin_size`.

```
plot_gene(  
  gene = "ERGIC1",  
  bam_file = sample_bam,  
  genome = "hg19",  
  bin_size = 10000  
)
```

10.6 Large BED or VCF files

For very large files, preprocessing or restricting the analysis to a chromosome or target region can reduce execution time.

11 Summary

GeGraphs provides a compact framework for combining genomic annotation, sequencing coverage and structural variant calls in the same analysis.

The package can be used to:

- inspect a candidate SV at gene level,
- compare coverage between a sample of interest and controls,
- visualize calls from several SV callers,
- explore genes located within or around an SV,
- intersect structural variants with a predefined gene panel,
- generate targeted figures for manual review.

The visualization is intended to support interpretation rather than replace variant validation or clinical assessment.

12 Citation

If you use GeGraphs in your work, please cite the package as:

Llugiqi, D. (2026). *GeGraphs: Gene-Centric Genomic Visualization Tools*. R package version 0.2.9.

Author: Dea Llugiqi

University of Geneva (UNIGE)

Master in Bioinformatics and Data Analysis in Biology (BIADB)

13 Session Info

The following information records the R environment used to build this tutorial and can be useful for reproducibility.

```
sessionInfo()
```

```

## R version 4.5.1 (2025-06-13 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=French_Switzerland.utf8 LC_CTYPE=French_Switzerland.utf8
## [3] LC_MONETARY=French_Switzerland.utf8 LC_NUMERIC=C
## [5] LC_TIME=French_Switzerland.utf8
##
## time zone: Europe/Zurich
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] GeGraphs_0.2.9
##
## loaded via a namespace (and not attached):
## [1] compiler_4.5.1    fastmap_1.2.0     cli_3.6.5         tools_4.5.1
## [5] htmltools_0.5.8.1 otel_0.2.0        rstudioapi_0.18.0 yaml_2.3.10
## [9] tinytex_0.58      rmarkdown_2.30    knitr_1.51        xfun_0.53
## [13] digest_0.6.37     rlang_1.2.0       evaluate_1.0.5

```